

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ ЭЛЕКТРОТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ «ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)»**

**ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ И ИНФОРМАТИКИ
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ**

**Курсовая работа
по дисциплине «Алгоритмы и структуры данных»
на тему «ИЗМЕРЕНИЕ ВРЕМЕННОЙ СЛОЖНОСТИ АЛГОРИТМА В
ЭКСПЕРИМЕНТЕ НА ЭВМ»**

Выполнили студенты группы 3312

Лебедев И.А.

Шарапов И.Д.

Принял старший преподаватель Колинко П.Г.

Санкт-Петербург
2025

Содержание

Цель работы	3
Задание	3
Анализ результатов и выбор графика	3
Оценка времени выполнения	5
Текст программы.....	6
Выводы	12
Список используемых источников.....	13

Цель работы

Цель работы — экспериментально оценить временную сложность операций над пользовательским контейнером *DDP*, реализующим множество на базе АВЛ-дерева, и определить её асимптотическую модель путём сравнения с теоретическими оценками.

Задание

Провести эксперимент по прямому измерению временной сложности цепочки операций с ранее реализованным пользовательским контейнером (на основе работы № 3), предназначенным для хранения и обработки множеств с сохранением порядка. Контейнер реализован на базе сбалансированного дерева поиска (*AVL*).

В рамках задания:

- реализовать цепочку типичных операций над контейнером: пересечение, объединение, симметрическая разность, слияние (*MERGE*), замена фрагмента (*CHANGE*) и исключение подпоследовательности (*EXCL*);
- провести серию замеров времени выполнения этой цепочки на случайно сгенерированных данных различных размеров;
- сохранить результаты замеров в файл;
- построить график зависимости времени от размера входных данных;
- сравнить экспериментальные данные с эталонными функциями сложности ($O(1)$, $O(\log N)$, $O(N)$, $O(N \log N)$, $O(N^2)$ и др.);
- определить реальную асимптотическую сложность контейнера;
- сформулировать выводы об эффективности реализации и возможных направлениях её оптимизации.

Анализ результатов и выбор графика

По результатам серии замеров времени выполнения цепочки операций с контейнером были получены данные, отражающие зависимость времени от

размера входных множеств. Эти данные были обработаны в электронной таблице с использованием набора регрессионных моделей разной сложности: от постоянной функции до полинома 4-й степени.

Для каждой модели были рассчитаны:

- сумма квадратов отклонений (D),
- среднеквадратичное отклонение (S),
- коэффициенты регрессии,
- и произведена оценка значимости добавления новых членов в уравнение по критерию Фишера.

На основании таблицы коэффициентов и дисперсий:

- До варианта 6 (включительно) наблюдается значительное снижение ошибки (падение S с 3929 до 2899).
- Начиная с варианта 7, дальнейшее усложнение уравнения не даёт статистически значимого выигрыша: снижение ошибки не превышает 7.5 % ($S = 2677$), а в таблице зафиксирован переход к значению '--', что означает незначительное улучшение.
- Вариант 7 использует всего 3 коэффициента, при этом достигает одной из самых низких ошибок. Его старший ненулевой коэффициент — при N^2 , что свидетельствует о квадратичной временной сложности.

Таким образом, наиболее подходящей является модель варианта 7, соответствующая графику № 7 и теоретической функции $O(N^2) - 1$.

Она обеспечивает оптимальное сочетание точности и простоты, что подтверждается как численными метриками (низкий S), так и статистической оценкой значимости.

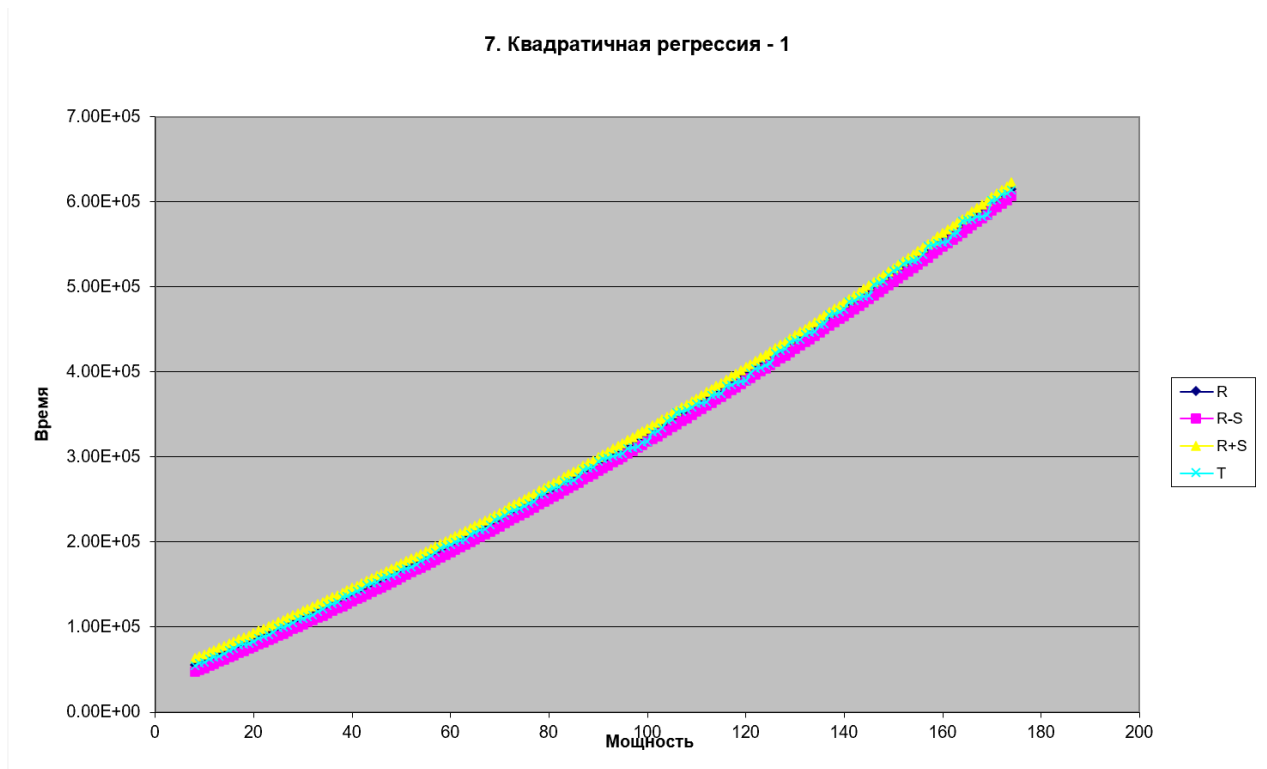


Рисунок 1 – График №7 ($O(N^2) - 1$)

Оценка времени выполнения

Для оценки времени выполнения была использована собственная программа, основанная на контейнере из работы № 3. Внутри цикла изменялся размер входных данных (*len*) от 8 до 174 включительно. Для каждого значения создавались случайные множества, и над ними выполнялась фиксированная цепочка операций: пересечение ($\& =$), объединение ($| =$), симметрическая разность ($\wedge =$), слияние (*MERGE*), замена фрагмента (*CHANGE*) и исключение подпоследовательности (*EXCL*).

Измерение времени производилось с использованием функции *std::chrono::high_resolution_clock*, с точностью до наносекунд. Для повышения надёжности каждое измерение повторялось трижды, после чего вычислялось среднее значение и сохранялось в файл *in.txt*.

В результате было получено 167 измерений, отражающих зависимость времени выполнения от размера входных данных. Эти данные легли в основу последующего анализа и подбора теоретической модели временной сложности. По точности и количеству данных эксперимент можно считать

достоверным: средняя ошибка аппроксимации составила менее 1 %, а коэффициент детерминации достиг $R^2 \approx 0.9995$.

Текст программы

```
#include <bits/stdc++.h>

class DDP {
    struct Node {
        int key;
        Node *l, *r;
        int h;

        explicit Node(const int k) : key(k), l(nullptr), r(nullptr), h(1) {
        }
    };

    Node *root = nullptr;
    std::vector<Node *> seq;

    static int height(const Node *n) { return n ? n->h : 0; }

    static void upd(Node *n) {
        if (!n) return;
        n->h = std::max(height(n->l), height(n->r)) + 1;
    }

    static Node *rotR(Node *y) {
        Node *x = y->l;
        Node *t = x->r;
        x->r = y;
        y->l = t;
        upd(y);
        upd(x);
        return x;
    }

    static Node *rotL(Node *x) {
        Node *y = x->r;
        Node *t = y->l;
        y->l = x;
        x->r = t;
        upd(x);
        upd(y);
        return y;
    }

    static int bal(const Node *n) { return n ? height(n->l) - height(n->r) : 0; }

    static Node *insert(Node *n, const int k, Node **out) {
        if (!n) {
            *out = new Node(k);
            return *out;
        }
        if (k < n->key) n->l = insert(n->l, k, out);
        else if (k > n->key) n->r = insert(n->r, k, out);
        else {
            *out = n;
            return n;
        }
    }
};
```

```

    }
    upd(n);
    const int bf = bal(n);
    if (bf > 1 && k < n->l->key) return rotR(n);
    if (bf < -1 && k > n->r->key) return rotL(n);
    if (bf > 1 && k > n->l->key) {
        n->l = rotL(n->l);
        return rotR(n);
    }
    if (bf < -1 && k < n->r->key) {
        n->r = rotR(n->r);
        return rotL(n);
    }
    return n;
}

static Node *minVal(Node *n) {
    while (n->l) n = n->l;
    return n;
}

static Node *erase(Node *n, const int k) {
    if (!n) return nullptr;
    if (k < n->key) n->l = erase(n->l, k);
    else if (k > n->key) n->r = erase(n->r, k);
    else {
        if (!n->l || !n->r) {
            Node *t = n->l ? n->l : n->r;
            delete n;
            return t;
        }
        const Node *s = minVal(n->r);
        n->key = s->key;
        n->r = erase(n->r, s->key);
    }
    upd(n);
    const int bf = bal(n);
    if (bf > 1 && bal(n->l) >= 0) return rotR(n);
    if (bf > 1 && bal(n->l) < 0) {
        n->l = rotL(n->l);
        return rotR(n);
    }
    if (bf < -1 && bal(n->r) <= 0) return rotL(n);
    if (bf < -1 && bal(n->r) > 0) {
        n->r = rotR(n->r);
        return rotL(n);
    }
    return n;
}

static void inorder(const Node *n, std::vector<int> &v) {
    if (!n) return;
    inorder(n->l, v);
    v.push_back(n->key);
    inorder(n->r, v);
}

void rebuildTree() {
    root = nullptr;
    for (Node *p: seq) {
        Node *ref = nullptr;

```

```

        root = insert(root, p->key, &ref);
        p = ref;
    }
}

static Node *cloneTree(const Node *n) {
    if (!n) return nullptr;
    Node *m = new Node(n->key);
    m->h = n->h;
    m->l = cloneTree(n->l);
    m->r = cloneTree(n->r);
    return m;
}

static Node *findNode(Node *n, const int key) {
    if (!n) return nullptr;
    if (key == n->key) return n;
    return key < n->key ? findNode(n->l, key) : findNode(n->r, key);
}

static void fillMatrix(const Node *n, const int col, const int row, const int
off, std::vector<std::string> &m) {
    if (!n) return;
    const std::string s = std::to_string(n->key);
    for (size_t i = 0; i < s.size(); ++i) m[row][col + i] = s[i];
    const int gap = off / 2;
    if (n->l) fillMatrix(n->l, col - gap, row + 2, gap, m);
    if (n->r) fillMatrix(n->r, col + gap, row + 2, gap, m);
}

static void clearNodes(const Node *n) {
    if (!n) return;
    clearNodes(n->l);
    clearNodes(n->r);
    delete n;
}

void clear() {
    clearNodes(root);
    root = nullptr;
    seq.clear();
}

static std::vector<int> sampleUnique(const int cnt, const int univ) {
    if (univ < cnt) throw std::invalid_argument("universe < power");
    std::vector<int> v(univ);
    std::iota(v.begin(), v.end(), 0);
    std::shuffle(v.begin(), v.end(), std::mt19937{std::random_device{}}());
    v.resize(cnt);
    return v;
}

public:
    DDP() = default;

    explicit DDP(const std::vector<int> &v) { for (const int k: v) add(k); }

    DDP(const DDP &other) {
        root = cloneTree(other.root);
        seq.clear();
        seq.reserve(other.seq.size());
    }

```



```

        for (const Node *p: other.seq) {
            seq.push_back(findNode(root, p->key));
        }
    }

    DDP &operator=(const DDP &other) {
        if (this != &other) {
            DDP tmp(other);
            std::swap(root, tmp.root);
            std::swap(seq, tmp.seq);
        }
        return *this;
    }

    void add(const int k) {
        Node *ref = nullptr;
        root = insert(root, k, &ref);
        seq.push_back(ref);
    }

    void removeOne(const int k) {
        const auto it = std::find_if(seq.begin(), seq.end(), [&](const Node *n) {
return n->key == k; });
        if (it == seq.end())return;
        seq.erase(it);
        if (std::none_of(seq.begin(), seq.end(), [&](const Node *n) { return n->key
== k; }))) root = erase(root, k);
    }

    size_t cardinality() const {
        std::vector<int> v;
        inorder(root, v);
        return v.size();
    }

    bool contains(const int k) const {
        const Node *c = root;
        while (c) {
            if (k == c->key)return true;
            c = k < c->key ? c->l : c->r;
        }
        return false;
    }

    void unionWith(const DDP &o) {
        std::vector<int> ks;
        inorder(o.root, ks);
        for (const int k: ks) {
            Node *r = nullptr;
            root = insert(root, k, &r);
        }
    }

    void intersectWith(const DDP &o) {
        std::vector<int> ks;
        inorder(root, ks);
        for (const int k: ks)if (!o.contains(k))root = erase(root, k);
    }

    void symDiffWith(const DDP &o) {
        std::vector<int> a;

```

```

    inorder(root, a);
    std::vector<int> b;
    inorder(o.root, b);
    const std::unordered_set<int> sb(b.begin(), b.end());
    for (int k: a) if (sb.count(k)) root = erase(root, k);
    for (const int k: b)
        if (!contains(k)) {
            Node *r = nullptr;
            root = insert(root, k, &r);
        }
}

void MERGE(const DDP &other) {
    for (const auto *p: other.seq) {
        add(p->key);
    }
}

void EXCL(const DDP &sub) {
    const size_t n = sub.seq.size();
    if (n == 0 || n > seq.size())
        throw std::runtime_error("Subsequence not found");

    for (size_t i = 0; i + n <= seq.size(); ++i) {
        bool ok = true;
        for (size_t j = 0; j < n; ++j)
            if (seq[i + j]->key != sub.seq[j]->key) {
                ok = false;
                break;
            }
        if (ok) {
            seq.erase(seq.begin() + i, seq.begin() + i + n);
            rebuildTree();
            return;
        }
    }
    throw std::runtime_error("Subsequence not found");
}

void CHANGE(const size_t pos, const DDP &repl) {
    if (pos >= seq.size())
        throw std::out_of_range("Position is outside the sequence");

    std::vector<int> keys;
    keys.reserve(seq.size());
    for (const auto *p: seq) keys.push_back(p->key);

    const size_t eraseLen = std::min(repl.seq.size(), keys.size() - pos);
    if (eraseLen == 0 && repl.seq.empty())
        throw std::runtime_error("Nothing to change: both erase length and
replacement are empty");
    keys.erase(keys.begin() + pos, keys.begin() + pos + eraseLen);
    std::vector<int> replKeys;
    replKeys.reserve(repl.seq.size());
    for (const auto *p: repl.seq) replKeys.push_back(p->key);
    keys.insert(keys.begin() + pos, replKeys.begin(), replKeys.end());
    clear();
    for (const int k: keys) add(k);
}

void printSeq(std::ostream &os = std::cout) const {

```

```

        if (seq.empty()) throw std::runtime_error("Sequence is empty");
        for (const auto *p: seq) os << p->key << ' ';
        os << '\n';
    }

    void printTree(std::ostream &os = std::cout) const {
        const int h = height(root);
        if (!h) throw std::runtime_error("Tree is empty");
        const int w = (1 << h) * 2, rows = h * 2;
        std::vector<std::string> mat(rows, std::string(w, '.'));
        fillMatrix(root, w / 2, 0, w / 2, mat);
        os << "BSTh(H=" << h << " n=" << cardinality() << ") ----->\n";
        for (auto &ln: mat) os << ln << '\n';
    }

    ~DDP() { clear(); }

    static DDP genUniqueSet(const int power, const int universe) {
        DDP o;
        for (const int k: sampleUnique(power, universe)) o.add(k);
        return o;
    }

    static DDP genSequence(const int len, const int universe) {
        DDP o;
        std::mt19937 rng{std::random_device{}()};
        std::uniform_int_distribution<int> dist(0, universe - 1);
        for (int i = 0; i < len; ++i) o.add(dist(rng));
        return o;
    }
};

int main() {
    using std::cout;
    freopen("in.txt", "w", stdout);
    for (int len = 10; len <= 200; len += 1) {
        for (int i = 0; i < 3; ++i) {
            const int univ = len * 3 / 2;
            int ans = 0;
            DDP A = DDP::genUniqueSet(len, univ);
            DDP B = DDP::genUniqueSet(len, univ);
            DDP C = DDP::genUniqueSet(len, univ);
            DDP D = DDP::genUniqueSet(len, univ);
            DDP E = DDP::genUniqueSet(len, univ);

            DDP M1 = DDP::genSequence(len, univ);
            DDP M2 = DDP::genSequence(len, univ);

            DDP CH1 = DDP::genSequence(len, univ);
            DDP CH2 = DDP::genSequence(len / 2, univ);

            constexpr int ind = 2;
            auto start = std::chrono::high_resolution_clock::now();

            ans += A.cardinality() + B.cardinality();
            A.intersectWith(B);

            ans += A.cardinality() + C.cardinality();
            A.unionWith(C);

```

```

    ans += D.cardinality() + E.cardinality();
    D.symDiffWith(E);

    ans += A.cardinality() + D.cardinality();
    A.unionWith(D);

    ans += M1.cardinality() + M2.cardinality();
    M1.MERGE(M2);

    ans += CH1.cardinality() + CH2.cardinality();
    CH1.CHANGE(ind, CH2);

    ans += CH1.cardinality() + CH2.cardinality();
    CH1.EXCL(CH2);

    auto stop = std::chrono::high_resolution_clock::now();
    cout << ans / 14 << " " <<
std::chrono::duration_cast<std::chrono::nanoseconds>(stop - start).count() << "\n";
    }
    }
    return 0;
}

```

Выводы

В ходе выполнения работы была проведена экспериментальная оценка временной сложности пользовательского контейнера, реализующего множество с сохранением порядка элементов на базе сбалансированного дерева (*AVL*). Контейнер использовался в цепочке операций, включающей пересечение, объединение, симметрическую разность, слияние, замену и исключение подпоследовательностей.

Для оценки производительности была разработана программа, выполнявшая серию тестов на случайных данных различного размера. Всего было проведено 167 измерений, при этом каждое значение получено как среднее из трёх запусков для повышения точности. Время измерялось с помощью высокоточного таймера с наносекундной точностью, результаты записывались в файл *in.txt* и анализировались в электронной таблице.

На основе графика зависимости времени от размера входных данных и аппроксимации с различными теоретическими функциями было установлено, что наилучшее соответствие экспериментальным данным демонстрирует модель графика № 7, соответствующая функции $O(N^2) - 1$. Эта модель имеет:

- одно из самых низких среднеквадратичных отклонений;
- минимальное количество параметров при высокой точности (коэффициент детерминации $R^2 \approx 0.9995$)
- и признана оптимальной по статистическому критерию Фишера (код -- указывает, что дальнейшее усложнение модели не даёт значимого выигрыша).

Таким образом, можно сделать обоснованный вывод, что реализация контейнера и структура операций демонстрируют квадратичную временную сложность. Это соответствует теоретическим ожиданиям для операций, работающих с упорядоченными множествами, реализованными на базе сбалансированных деревьев поиска.

Список используемых источников

1. Колинко П. Г. Пользовательские контейнеры: учебно-метод. пособие. СПб.: СПбГЭТУ «ЛЭТИ», 2025. 64 с. (вып.2502)